R.M.K GROUP OF ENGINEERING INSTITUTIONS

R.M.K
GROUP OF INSTITUTIONS

R.M.K
GROUP OF
INSTITUTIONS

R.M.K GROUP OF INSTITUTIONS

Please read this disclaimer before proceeding:

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail. Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.

R.M.K GROUP OF INSTITUTIONS

22CS201 Data Structures

Department : **CSE , IT , AIML**

Batch / Year : 2023-2027 / I

Created by : Subject Handling Staff Members

Date :11.03.2024

R.M.K GROUP OF INSTITUTIONS

Table of Contents

Course Objectives

22CS201 Data Structures

L T P C
3 0 2 4

- **Course Objectives:**
- To understand the concepts of List ADT.
- To learn linear data structures – stacks and queues ADTs.
- To understand and apply Tree data structures.
- To understand and apply Graph structures.
- To analyze sorting, searching and hashing algorithms.

R.M.K GROUP OF INSTITUTIONS

Pre Requisites (Course Names with Code)

- 22CS101 - Programming using C++

R.M.K GROUP OF INSTITUTIONS

4. SYLLABUS

22CS201 **DATA STRUCTURES** **L T P C**
3 0 2 4

OBJECTIVES

- To understand the concepts of List ADT.
- To learn linear data structures – stacks, and queues ADTs.
- To understand and apply Tree data structures
- To understand and apply Graph structures
- To analyze sorting, searching and hashing algorithms

UNIT I - LINEAR DATA STRUCTURES – LIST **9**

Algorithm analysis-running time calculations-Abstract Data Types (ADTs) – List ADT – array-based implementation – linked list implementation – singly linked lists- circularly linked lists- doubly-linked lists – applications of lists –Polynomial Manipulation – All operations (Insertion, Deletion, Merge, Traversal).

List of Exercise/Experiments:

1. Array implementation of List, Stack and Queue ADTs.
2. Linked list implementation of List, Stack and Queue ADTs.
3. Applications of List – Polynomial manipulations

UNIT II - LINEAR DATA STRUCTURES – STACKS, QUEUES **9**

Stack ADT – Stack Model - Implementations: Array and Linked list – Applications Balancing symbols - Evaluating arithmetic expressions - Conversion of Infix to postfix expression- Queue ADT – Queue Model - Implementations: Array and Linked list - applications of queues - Priority Queues – Binary Heap – Applications of Priority Queues.

List of Exercise/Experiments:

1. Applications of Stack – Infix to postfix conversion and expression evaluation.
2. Implementation of Heaps using Priority Queues.

UNIT III - NON LINEAR DATA STRUCTURES – TREES **9**

Tree ADT – tree traversals - Binary Tree ADT – expression trees – applications of trees – binary search tree ADT– AVL Tree.

List of Exercise/Experiments:

1. Implementation of Binary Trees and operations of Binary Trees.
2. Implementation of Binary Search Trees.

UNIT IV - NON LINEAR DATA STRUCTURES – GRAPHS **9**

Definition – Representation of Graph – Types of graph - Breadth-first traversal - Depth-first traversal – Topological Sort – Applications of graphs – BiConnectivity – Euler circuits.

List of Exercise/Experiments:

1. Graph representation and Traversal algorithms.

UNIT V - SEARCHING, SORTING AND HASHING TECHNIQUES **9**

Searching- Linear Search - Binary Search - Sorting - Bubble sort - Selection sort - Insertion sort – Hashing- Hash Functions – Separate Chaining – Open Addressing – Rehashing – Extendible Hashing.

List of Exercise/Experiments:

1. Implement searching and sorting algorithms.

TOTAL: 45+ 30 = 75 PERIODS

Course Outcomes

- At the end of this course, the students will be able to:
- CO1: Implement abstract data types for list.
- CO2: Solve real world problems using appropriate linear data structures.
- CO3: Apply appropriate tree data structures in problem solving.
- CO4: Implement appropriate Graph representations and solve real-world applications.
- CO5: Implement various searching and sorting algorithms.

6. CO - PO / PSO MAPPING

CO	HKL	PROGRAM OUTCOMES												PSO		
		K3	K4	K5	K5	K3, K4, K5	A3	A2	A3	A3	A3	A3	A2	PSO1	PSO2	PSO3
		PO-1	PO-2	PO-3	PO-4	PO-5	PO-6	PO-7	PO-8	PO-9	PO-10	PO-11	PO-12			
C203.1	K3	3	2	1												
C203.2	K3	3	2	1												
C203.3	K3	3	2	1												
C203.4	K3	3	2	1												
C203.5	K3	3	2	1												

- **Correlation Level - 1. Slight (Low) 2. Moderate (Medium)**
  3. Substantial (High), If there is no correlation, put "-".

Lecture Plan

Unit IV

Unit IV

S. No.	Topic	No. of Periods	Proposed Date	Actual Lecture Date	Pertaining CO	Highest Cognitive Level	Mode of Delivery
1	Graph-Definition, Representation	1	27.03.2024	27.03.2024	CO4	K2	Oral presentation
2	Types of Graph, Breadth First Traversal	1	28.03.2024	28.03.2024	CO4	K3	Oral presentation
3	Depth First Traversal	1	01.04.2024	01.04.2024	CO4	K3	Oral presentation
4	Topological Sort	1	02.04.2024	02.04.2024	CO4	K3	videos
5	Applications of graphs	1	03.04.2024	03.04.2024	CO4	K2	Tutorial
6	Bi - connectivity	1	04.04.2024	04.04.2024	CO4	K3	seminar
7	Euler circuits	1	05.04.2024	05.04.2024	CO4	K3	Oral presentation

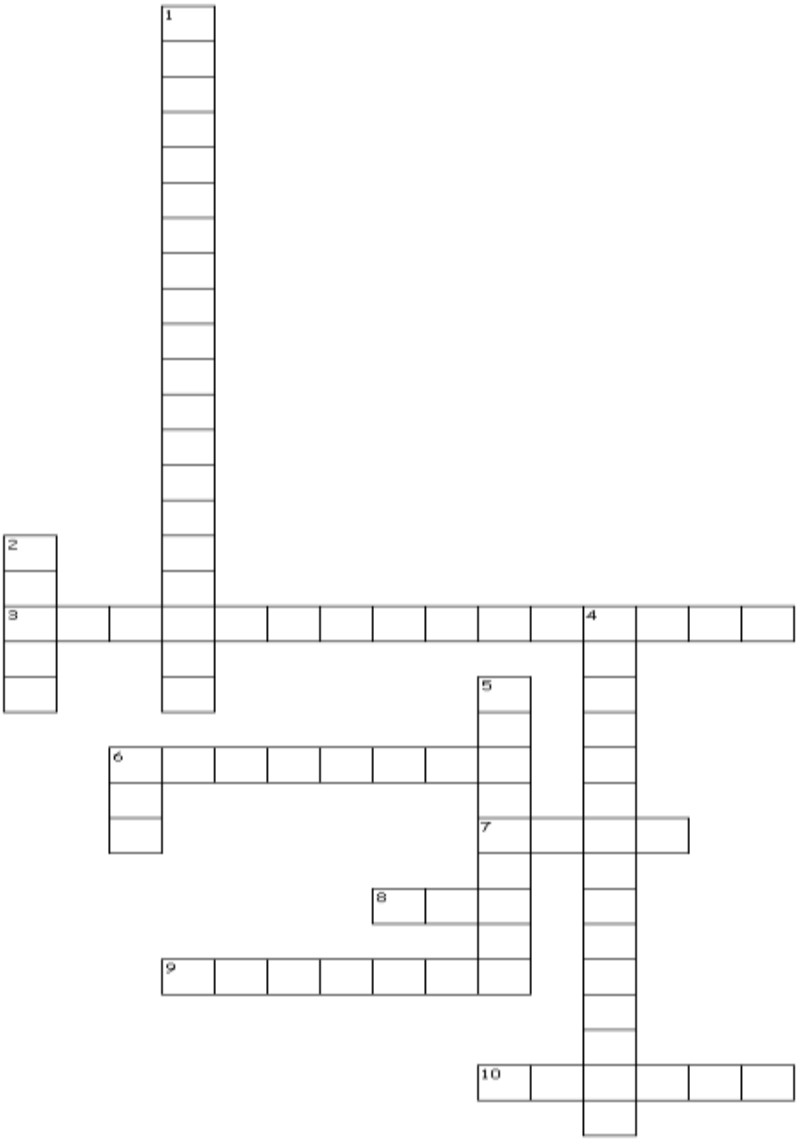
Activity Based Learning

Unit IV

R.M.K
GROUP OF
INSTITUTIONS

8. ACTIVITY BASED LEARNING : UNIT – IV

CROSSWORD PUZZLE ON GRAPHS



R.M.K GROUP OF INSTITUTIONS

8. ACTIVITY BASED LEARNING : UNIT – IV

CROSSWORD PUZZLE ON GRAPHS – CLUES

Across

3. representation of graph

6. shortest path

7. path from one vertex to another

8. uses queue

9.minimum spanning tree

10.node in a graph

Down

1. no cycles

2. vertices and edges

4. an ordering of vertices

5. visit a vertex exactly once

6. uses stack

Lecture Notes

UNIT IV - NON LINEAR DATA STRUCTURES – GRAPHS

Definition – Representation of Graph – Types of graph - Breadth-first traversal - Depth-first traversal – Topological Sort – Applications of graphs – Bi Connectivity – Euler circuits.

GRAPH ADT

4.1 INTRODUCTION

9. LECTURE NOTES : UNIT – IV

NON LINEAR DATA STRUCTURES-GRAPHS

GRAPHS

Definition

A graph G=(V,E) consists of a set of vertices V and a set of edges, E. Each edge is a pair(u,v) where u,v ε V

- Edges are sometimes referred to as arcs
- Vertices are referred as nodes

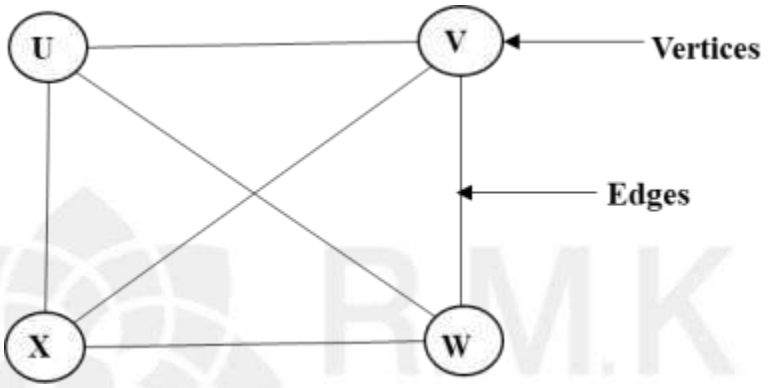


Fig 4:1

Types of graphs

Directed graph or Digraph

It is a graph which consists of directed edges where each edge in E is unidirectional. If (u,v) is a directed edge, then (u,v) ≠ (v,u).

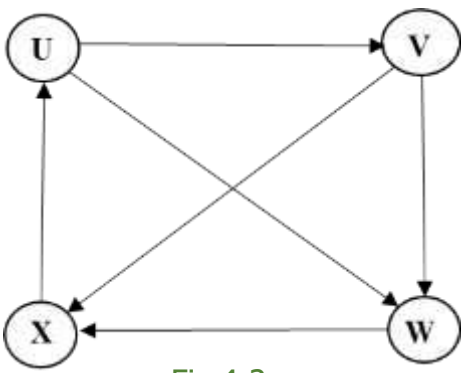


Fig 4:2

Undirected Graph

It consist of undirected edges. Here (u,v)=(v,u)

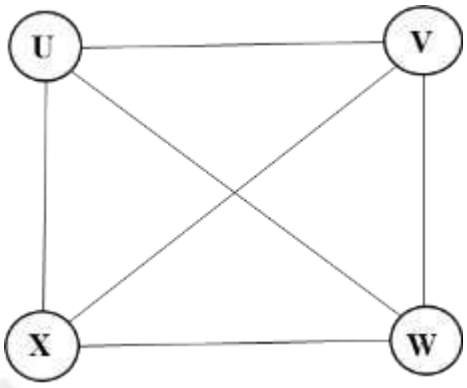


Fig 4:3

Weighted Graph

- There is another component that is associated with edge, which is known as weight or cost.
- A graph is said to be weighted graph if every edge in the graph is assigned a weight or cost. The weight can be assigned both the directed and undirected graph.

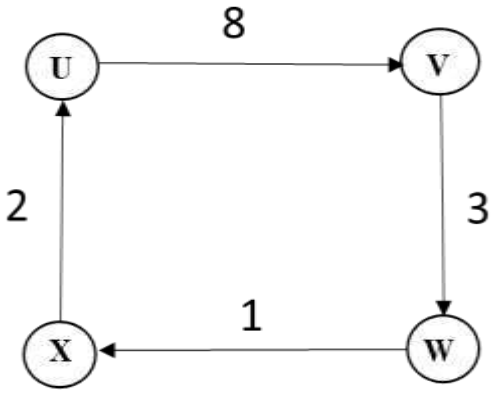


Fig 4:4

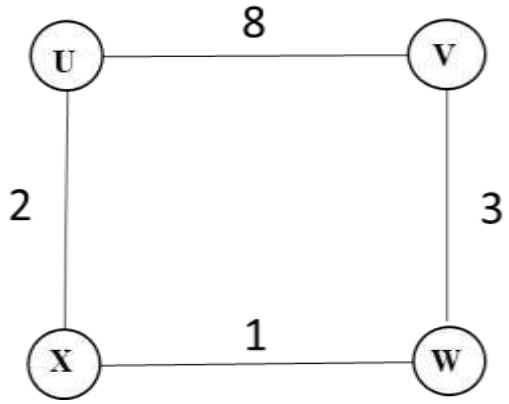


Fig 4:5

Path

A path in a graph is a sequence of vertices $w_1,w_2,w_3,.....w_n$ such that $(wi,w_{i+1})\varepsilon$ E. For $1 \leq i < N$. The path from U to X is UVWX.

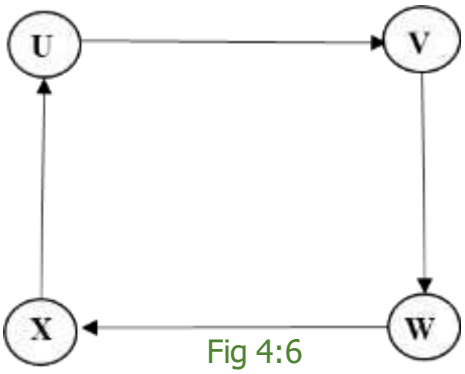


Fig 4:6

Length

Length of the path is the number of edges on the path, which is equal to N-1,where N is the number of vertices.

The length of the path from U to X is 3.

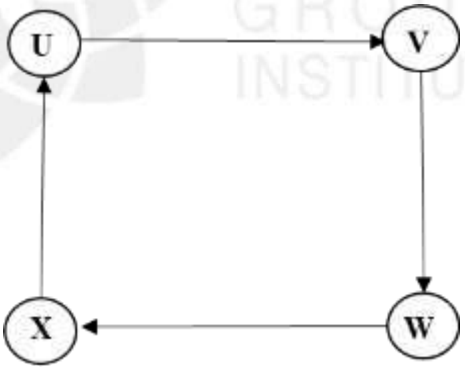


Fig 4:7

When there is a path from a vertex to itself and if the path contains no edge, then the path length is '0'.

Loop

If the graph contains an edge (u,v) from a vertex to itself, then the path u,v is referred to as a loop. In Fig 4.7.1 Vertices a has the loop to itself.

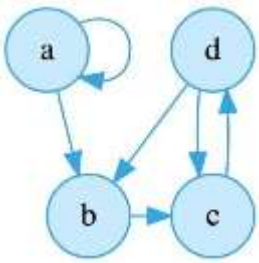


Fig 4:7:1

Simple Path

Simple path is a path such that all vertices are distinct, expect that the first and last could be same.

Cycle

A cycle in a directed graph is a path in which first and last vertex are the same.

Cyclic Graph

A graph which has cycles is referred to as cyclic graph.

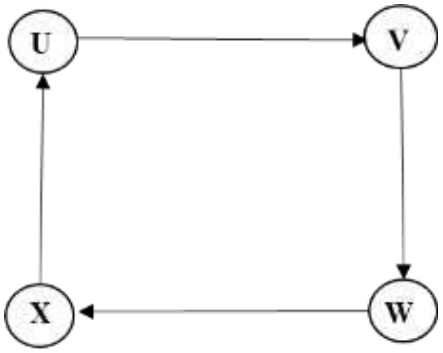


Fig 4:8

Acyclic Graph

A directed graph is acyclic if it has no cycles. A directed acyclic graph is sometimes referred as DAG

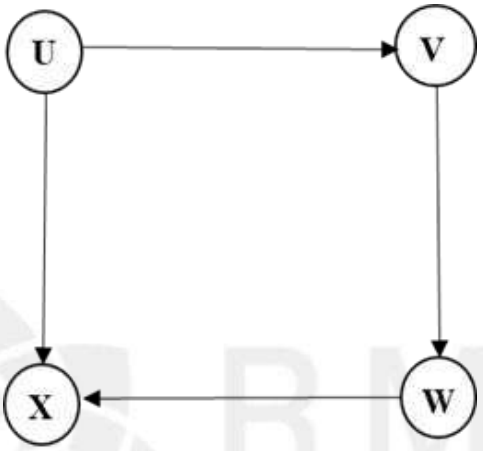


Fig 4:9

Complete Graph

A complete graph is a graph in which there is an edge between every pair of vertices. A complete graph with n vertices will have n(n-1)/2 edges.

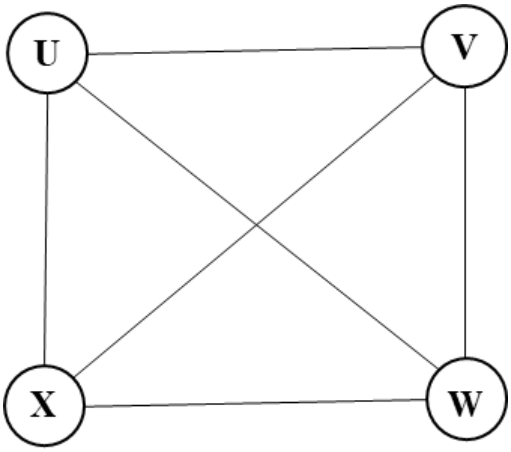


Fig 4:10

Strongly Connected Graph

If there is a path from every vertex to every other vertex in a directed graph then it is said to be strongly connected graph.

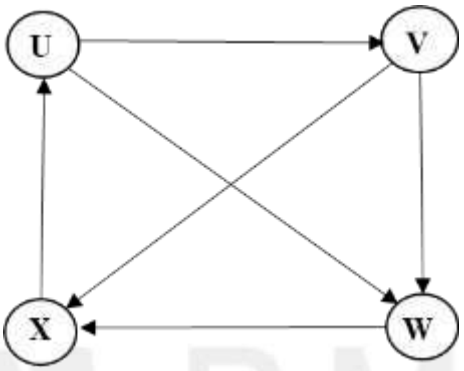


Fig 4:11

Weakly Connected Graph

In a directed graph if it is connected but there is no path from every vertex to every other vertex, then it is said to be weakly connected graph.

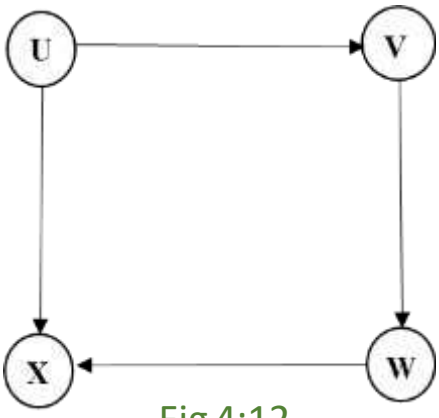


Fig 4:12

Degree

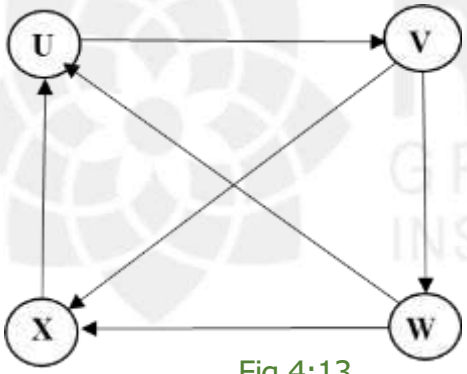
The number of edges incident on a vertex determines its degree.

Indegree

Indegree of the vertex 'V' is the number of edges entering into the vertex V.

Outdegree

Outdegree of the vertex 'V' is the number of edges exiting from that vertex V.



Indegree(V)=1

Outdegree(V)=2

Fig 4:13

Sink Node

A node that does not have any outgoing edges (i.e.) only indegree and outdegree will be 0.

Source Node

A node that does not have any incoming edges (i.e.) only outdegree and indegree will be 0.

Representation of Graph

Graph can be represented in two ways

1. Adjacency Matrix
2. Adjacency List

Adjacency Matrix

- One simple way to represent a graph is to use a 2-D array. This is Known as adjacency matrix representation.
- The adjacency matrix A for a graph G=(V,E) with n vertices is a nxn matrix,such that Aij=1 if there is an edge between Vi and Vj

  Aij=0 if there is no edge.

Example : Adjacency matrix for directed graph.

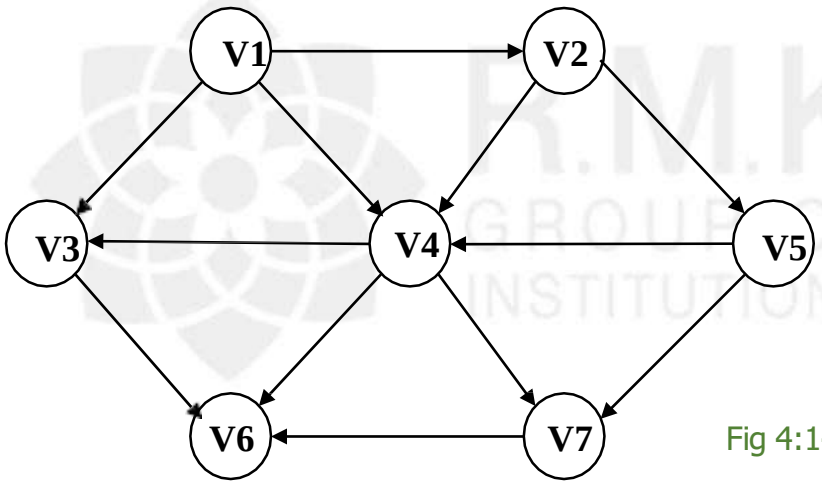


Fig 4:14

	V1	V2	V3	V4	V5	V6	V7
V1	0	1	1	1	0	0	0
V2	0	0	0	1	1	0	0
V3	0	0	0	0	0	1	0
V4	0	0	1	0	0	1	1
V5	0	0	0	1	0	0	1
V6	0	0	0	0	0	0	0
V7	0	0	0	0	0	0	0

- If the edge has a weight associated with it, then we can set Aij equal to the weight.

  (i.e.) Aij=Cij, if there exists an edge from Vi to Vj.
- Otherwise use either a very large or a very small weight to indicate the non existent edges.
- Adjacency matrix is an appropriate representation if the graph is dense.

Adjacency List Representation

- If the graph is sparse, a better way to represent graph is an adjacency list.
- All the vertices are stored in a list and then for each vertex, all its adjacent vertex are kept in a list.

Example : Adjacency List for directed graph

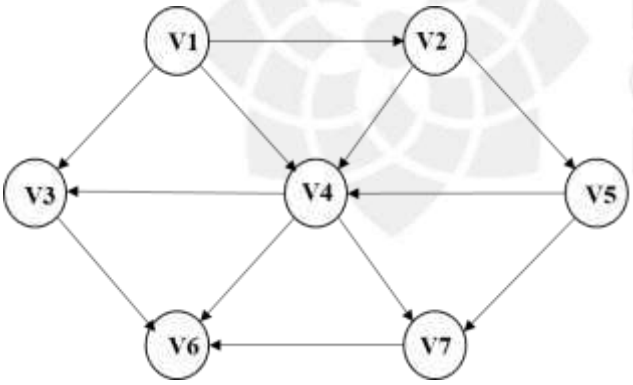


Fig 4:15 (a)

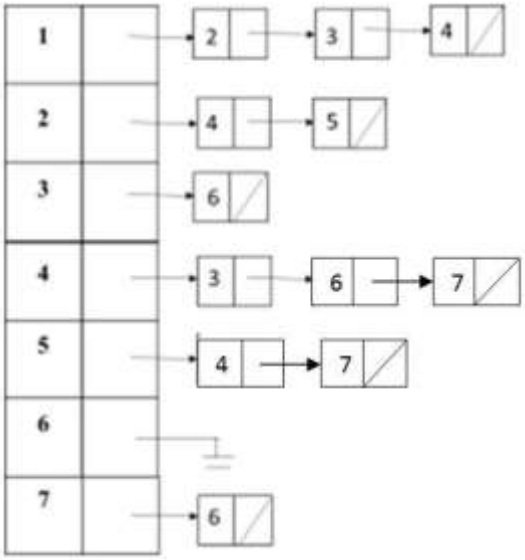


Fig 4:15 (b)

9. LECTURE NOTES : UNIT – IV

Depth First Traversal/Search

- DFS is a generalization of preorder traversal.
- DFS works by selecting one vertex V of G as a start vertex, process V and then recursively traverse all vertices adjacent to V.
- DFS algorithm traverses a graph in a depthward motion and uses a stack to backtrack any vertex, when a dead end occurs.
- It employs the following rules

Rule 1 :Visit the adjacent unvisited vertex . Mark it as visited. Display it.Push it in a stack.

Rule 2:If no adjacent vertices is found. Pop up a vertex from the stack.

Rule 3:Repeat Rule1 and Rule2 until the stack is empty.

Example :

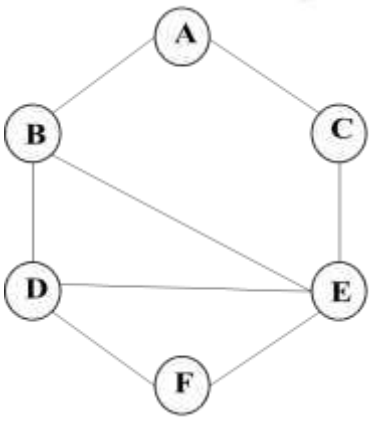


Fig 4:16

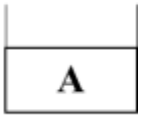
	A	B	C	D	E	F
A	0	1	1	0	0	0
B	1	0	0	1	1	0
C	1	0	0	0	1	0
D	0	1	0	0	1	1
E	0	1	1	1	0	1
F	0	0	0	1	1	0

Initial visited array

A	B	C	D	E	F
0	0	0	0	0	0

1. Consider 'A' as the start vertex and visit it.

	A	B	C	D	E	F
1	1	0	0	0	0	0

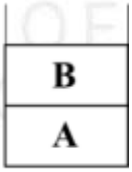


DFS O/P :A

Adjacent vertex to 'A' are 'B' and 'C'

2. Consider the next unvisited vertex which is B and visit it.

	A	B	C	D	E	F
2	1	1	0	0	0	0

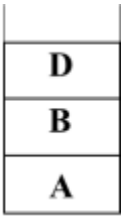


DFS O/P:AB

Adjacent vertex to 'B' are 'A','D' and 'E'

3. But the next unvisited vertex is 'D' so visit it

	A	B	C	D	E	F
3	1	1	0	1	0	0



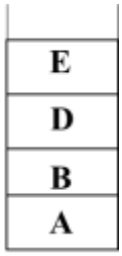
DFS O/P:ABD

Adjacent vertex to 'D' are 'B','E' AND 'F'

4. But the next unvisited vertex is 'E' so visit it

4

A	B	C	D	E	F
1	1	0	1	1	0



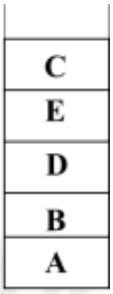
DFS O/P: A B D E

Adjacent vertex to 'E' are 'B','C','D' AND 'F'

5. But the next unvisited vertex is 'C' so visit it

5

A	B	C	D	E	F
1	1	1	1	1	0



DFS O/P A B D E C

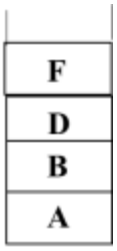
Adjacent vertex to 'C' are 'A', and 'E'

6. But both the vertex are already visited, so pop the vertices from the stack to find a vertex which has an unvisited vertex. Pop 'C'-All adjacent vertex visited.

Pop 'E'-Adjacent vertex 'F' not visited so visit it

6

A	B	C	D	E	F
1	1	1	1	1	1



DFS O/P ABDECF

Adjacent vertex of 'F' are 'D' and 'E', which is already visited.

Pop 'F'-All adjacent vertex visited

Pop 'D', Pop 'B',Pop 'A' and finally the stack is empty.

DFS O/P A B D E C F

Routine for Depth First Search

```
Void DFS(Vertex V)
{
        visited[ v ]=True;
                for each w adjacent to v
                 if(! Visited[ w ])
                        DFS(w);
 }
```

Breadth First Traversal/Search

- A Breadth First Search(BFS) algorithm traverses a graph in a breadth ward motion uses a queue to backtrack, when a dead end occurs.
- It employs the following rules

Rule 1:

Visit the adjacent unvisited vertex. Mark it as visited. Display it. Insert it in a Queue.

Rule 2:

If no adjacent vertex is found, remove the first vertex from the Queue.

Rule 3:

Repeat Rule 1 and Rule 2 until the queue is empty.

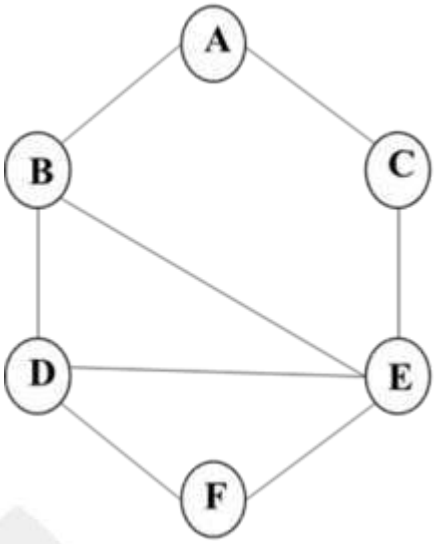
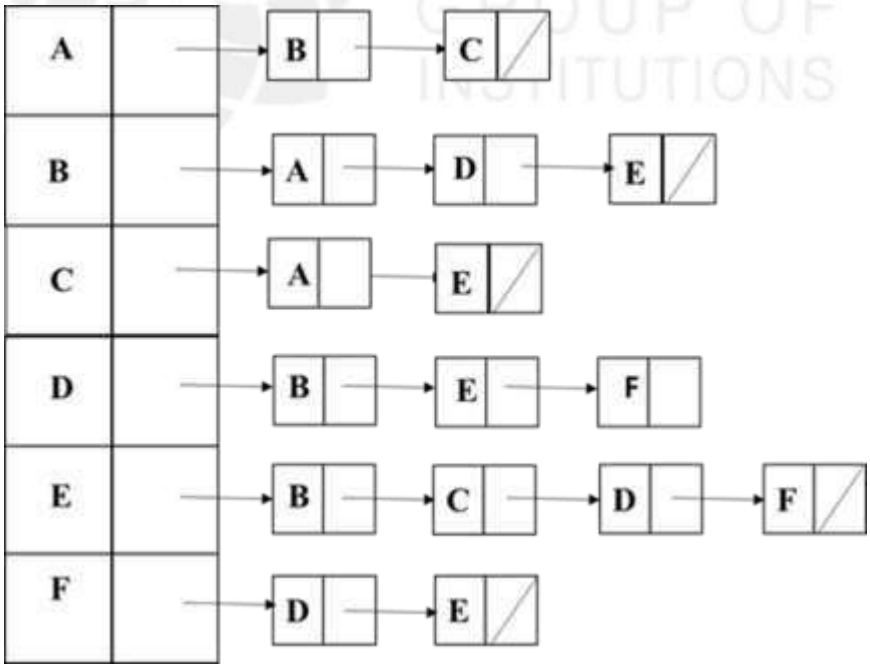


Fig 4:17



Initial visited array

A	B	C	D	E	F
0	0	0	0	0	0

1. Consider 'A' as the start vertex and visit it.

A	B	C	D	E	F
1	0	0	0	0	0

A			

BFS O/P : A

Adjacent vertex to 'A' are 'B' and 'C'

2. Remove 'A' from queue. Visit the adjacent vertex 'B' and 'C' and insert in the queue.

A	B	C	D	E	F
1	1	1	0	0	0

B	C		

Remove 'B' from Queue

BFS O/P : A B

Adjacent vertex to 'B' are 'A','D' and 'E'

3. The unvisited vertex are 'D' and 'E' ,Visit it and insert in the queue.

A	B	C	D	E	F
1	1	1	1	1	0

C	D	E	

Remove 'C' from Queue

BFS O/P : ABC

Adjacent vertex to 'C' are 'A' and 'E'

4. 'A' and 'E' are already visited. So remove 'D' from Queue.

A	B	C	D	E	F
1	1	1	1	1	0

E		

BFS O/P : A B C D

Adjacent vertex to 'D' are 'B','E' and 'F'.

5. 'B' and 'E' are already visited. Visit F and insert to Queue.

A	B	C	D	E	F
1	1	1	1	1	1

E	F	

Remove 'E' from Queue

BFS O/P : A B C D E

Adjacent vertex to 'E' are 'B','C','D' and 'F'

6. All are visited, so remove 'F' from Queue

A	B	C	D	E	F
1	1	1	1	1	1

BFS O/P : ABCDEF

The Queue is empty.

BFS O/P : A B C D E F

Routine for Breadth First Search

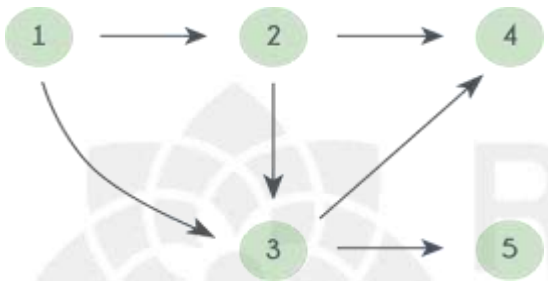
A Breadth First traversal visits all successors of visited node before visiting any successors of it.

```
void bfs(vertex V)
{
      vertex w , v;
       Enqueue(v);
       while(!isempty(Queue))
        {
            v=Dequeue();
            if(!visited[v])
                        cout<<v;
            visited[v]=TRUE;
            for each w adjacent to v
               if(visited[w]==FALSE)
                {
                        cout<<w;
                        visited[w]=TRUE
                        Enqueue(w);
                 }
          }
      }
```

Topological Sort

In computer science, a topological sort or topological ordering of a directed graph is a linear ordering of its vertices such that for every directed edge (u,v) from vertex u to vertex v, u comes before v in the ordering. For instance, the vertices of the graph may represent tasks to be performed, and the edges may represent constraints that one task must be performed before another.

Topological sorting of vertices of a **Directed Acyclic Graph** is an ordering of the vertices v1,v2,v3….vn in such a way that if there is an edge directed towards vertex vj , from vertex vi then vi comes before vj. For example consider the graph given below:



A topological sorting of this graph is: 1 2 3 4 5

There are multiple topological sorting possible for a graph.

For the graph given above one another topological sorting is: 1 2 3 5 4

In order to have a topological sorting the graph must not contain any cycles.

Topological sorting can be achieved for only directed and acyclic graphs.

Algorithm :

First the indegree is computed for every vertex.

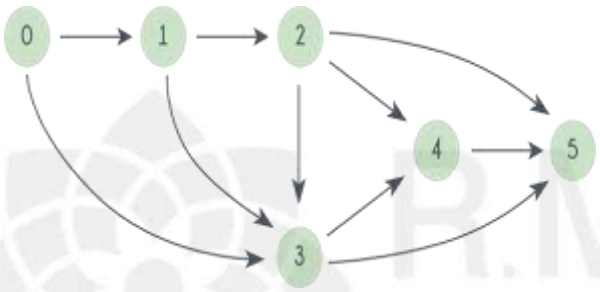
Then all the vertices of indegree 0 are placed on an initially empty queue.

While the queue is not empty, a vertex 'v' have the indegrees decremented.

A vertex is put on the queue as soon as its indegree falls to zero.

The topological ordering then is the order in which the vertices dequeued.

Consider the following Graph:



QUEUE : 0

in_degree

0	1	2	2	2	3
0	*1*	*2*	*3*	*4*	*5*

T : --

QUEUE : 2

in_degree

0	0	0	1	1	3
0	*1*	*2*	*3*	*4*	*5*

T : 0 1

QUEUE : 3

in_degree

0	0	0	0	1	2
0	*1*	*2*	*3*	*4*	*5*

T : 0 1 2

↓

QUEUE : 4

in_degree

0	0	0	0	0	1
0	*1*	*2*	*3*	*4*	*5*

T : 0 1 2 3

↓

QUEUE : 5

in_degree

0	0	0	0	0	0
0	*1*	*2*	*3*	*4*	*5*

T : 0 1 2 3 4

↓

QUEUE : --

in_degree

0	0	0	0	0	0
0	*1*	*2*	*3*	*4*	*5*

T : 0 1 2 3 4 5

Another Example:

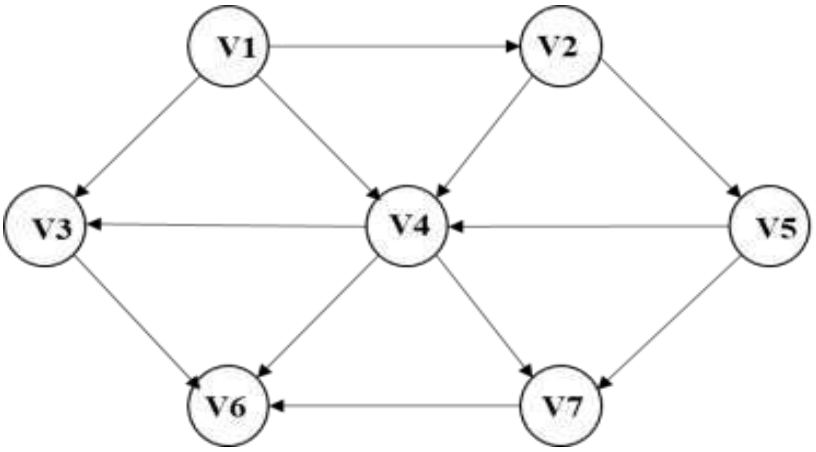


Fig 4:18

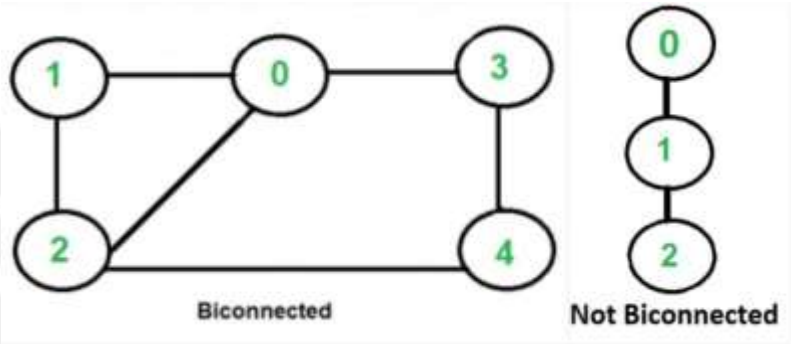
Vertex	V1	V2	V3	V4	V5	V6	V7
V1	0	0	0	0	0	0	0
V2	1	0	0	0	0	0	0
V3	2	1	1	1	0	0	0
V4	3	2	1	0	0	0	0
V5	1	1	0	0	0	0	0
V6	3	3	3	3	2	1	0
V7	2	2	2	1	0	0	0
Enqueue	V1	V2	V5	V4	V3,V7	----	V6
Dequeue	V1	V2	V5	V4	V3	V7	

Topological Ordering: V1, V2, V5, V4, V3, V7, V6

9. LECTURE NOTES : UNIT – IV

BICONNECTIVITY :

- A connected undirected graph is biconnective if there are no vertices whose removal disconnects the rest of the graph. A biconnected undirected graph is a connected graph that is not broken into disconnected pieces by deleting any single vertex (and its incident edges). A biconnected directed graph is one such that for any two vertices v and w there are two directed paths from v to w which have no vertices in common other than v and w. If a is not bi-connected, the vertices whose removal would disconnect the graph is called articulation points.

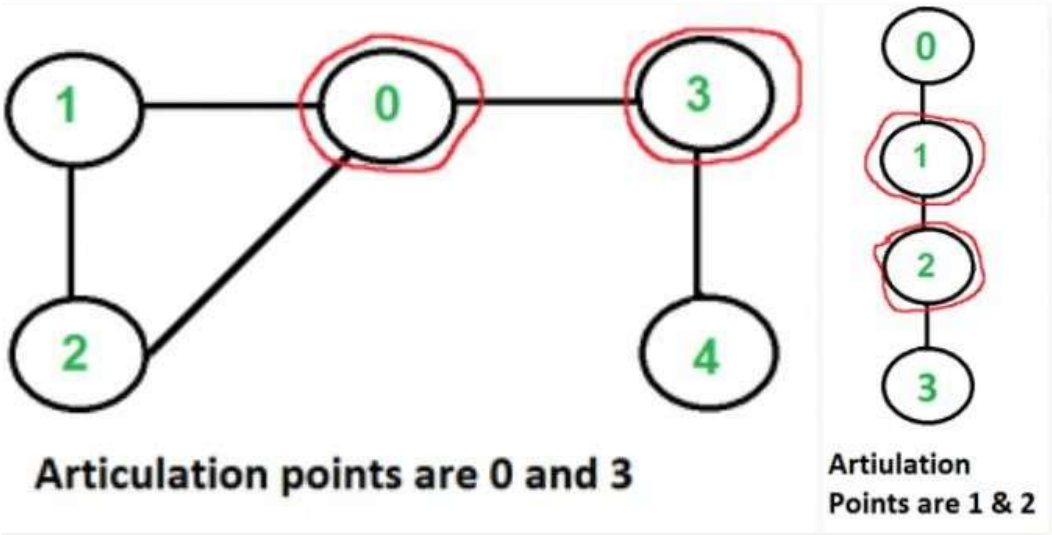


- DEFINITION - Equivalent definitions of a biconnected graph G: Graph G has no separation edges and no separation vertices For any two vertices u and v of G, there are two disjoint simple paths between u and v (i.e., two simple paths between u and v that share no other vertices or edges) For any two vertices u and v of G, there is a simple cycle containing u and v.

  Biconnected Components: Biconnected component of a graph G are: A maximal biconnected subgraph of G, or A subgraph consisting of a separation edge of G and its end vertices.

- Interaction of biconnected components : An edge belongs to exactly one biconnected component A nonseparation vertex belongs to exactly one biconnected component A separation vertex belongs to two or more biconnected components.

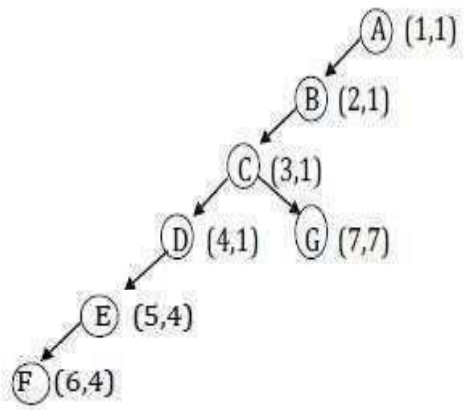
- **Articulation Point** : The vertices whose removal disconnects the graph are known as Articulation Points. Steps to find Articulation Points : (i) Perform DFS, starting at any vertex. (ii) Number the vertex as they are visited as Num(V). (iii) Compute the lowest numbered vertex for every vertex V in the DFS tree, which we call as low(W), that is reachable from V by taking one or more tree edges and then possible one back edge by definition.



Low(V) = min(Num(V), Num(W), Low(W))

The Lowest Num(W) among all back edges V, W. The Lowest Low(W) among all the tree edges V, W.

- The root is an articulation if and only if (iff) it has more than two children. Any vertex V
- other than the root is an Articulation point iff V has some child W such that **Low(W)≥Num (V)**



Low(F) = min(Num(V), Num(W), Low(W)) = min(6, 4, -1) = 4
Low(E) = min(5, 6, 4) = 4 Low(D) = min(4, 1, 4) = 1

Low(G) = min(7, -, -) = 7 Low(C) = min(3, (4,7), (1,7)) = 1 Low(B) = min(2, 3, 1) = 1 Low(A) = min(1, 2, 1) = 1

ALGORITHM

Routine to assign Num to Vertices

```
void AssignNum(Vertex V)
{
Vertex W;
int counter = 0;
Num[V] = ++counter;
visited[V] = True;
for each W adjacent to V if(!visited[W])
 {
parent[W] = V;
AssignNum(W);
 }
}
```

9. LECTURE NOTES : UNIT – IV

- **Routine to compute low and to test for points**

```
void AssignLow(Vertex V)
{
Vertex W; Low[V] = Num[V]; /* Rule 1 */ for each W adjacent to V {
if(Num[W] > Num[V]) /* forward edge or free edge */
{ AssignLow(W)  if(low[W] >=
Num[V])
   cout<<V;          Low[V] = min(Low[V], Low[W]); /* Rule
   3 */ }
else {
if(parent[V]! = W) /* Back edge */ Low[V] = min(Low[V], Num[W]); /* Rule 2 */
}
}
}
```

APPLICATION

- Bio-Connectivity is a application of depth first search.
- Used mainly in network concepts.

BICONNECTIVITY ADVANTAGES

- Total time to perform traversal is minimum.
- Adjacency lists are used
- Traversal is given by O(E+V).

DISADVANTAGES

- Have to be careful to avoid cycles , Vertices should be carefully removed as it affects the rest of the graph.

9. LECTURE NOTES : UNIT – IV

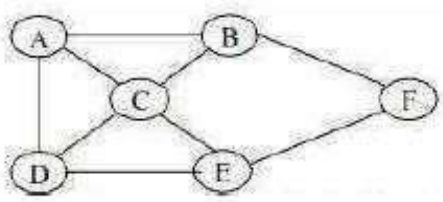
EULER CIRCUIT

EULERIAN PATH

An Eulerian path in an undirected graph is a path that uses each edge exactly once. If such a path exists, the graph is called traversable or semi- eulerian.

EULERIAN CIRCUIT

An Eulerian circuit or Euler tour in an undirected graph is a cycle that uses each edge exactly once. If such a cycle exists, the graph is called Unicursal. While such graphs are Eulerian graphs, not every Eulerian graph possesses an Eulerian cycle.



EULER'S THEOREM

Euler's theorem 1

If a graph has any vertex of odd degree then it cannot have an Euler circuit. If a graph is connected and every vertex is of even degree, then it at least has one Euler circuit.

Euler's theorem 2

If a graph has more than two vertices of odd degree then it cannot have an Euler path.

If a graph is connected and has just two vertices of odd degree, then it at least has one Euler path. Any such path must start at one of the odd- vertices and end at the other odd vertex.

ALGORITHM

Fleury's Algorithm for finding an Euler Circuit

1.Check to make sure that the graph is **connected and all vertices are of even degree**

2.Start at any vertex

3.Travel through an edge: 0 If it is **not a bridge for the untraveled part, or** 0 there is no other alternative

4. Label the edges in the order in which you travel them.

5. When you cannot travel any more, stop.

9. LECTURE NOTES : UNIT – IV

Fleury's Algorithm

- pick any vertex to start.
- From that vertex pick an edge to traverse.
- Darken that edge, as a reminder that you can't traverse it again.
- Travel that edge, coming to the next vertex.
- Repeat 2-4 until all edges have been traversed, and you are back at the starting vertex . At each stage of the algorithm: the original graph minus the darkened (already used) edges = **reduced graph**

important rule: ***never cross a bridge of the reduced graph unless there is*** no other choice

Note:

- The same algorithm works for Euler paths
- before starting, use Euler's theorems to check that the graph has an
- Euler path and/or circuit to find.

APPLICATION

Eulerian paths are being used in bioinformatics to reconstruct the DNA SEQUENCE from its fragments.

9. LECTURE NOTES : UNIT – IV

Application of Graphs

Kruskal's Minimum Spanning Tree Algorithm

Given a connected and undirected graph, a spanning tree of that graph is a subgraph that is a tree and connects all the vertices together. A single graph can
have many different spanning trees.

A minimum spanning tree (MST) or minimum weight spanning tree for a weighted, connected and undirected graph is a spanning tree with weight less than or equal to the weight of every other spanning tree.

The weight of a spanning tree is the sum of weights given to each edge of the spanning tree. How many edges does a minimum spanning tree has?

A minimum spanning tree has (V – 1) edges where V is the number of vertices in the given graph.

9. LECTURE NOTES : UNIT – IV

Below are the steps for finding MST using Kruskal's algorithm

Sort all the edges in non-decreasing order of their weight.

Pick the smallest edge. Check if it forms a cycle with the spanning

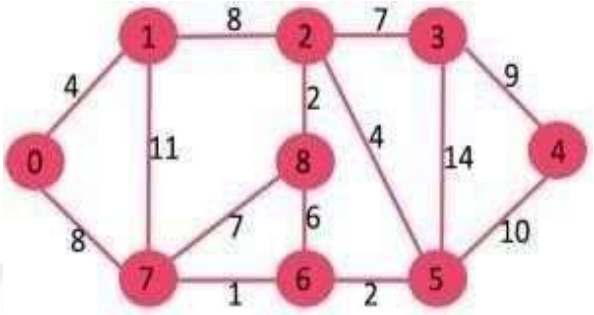
- tree formed so far. If

  cycle is not formed, include this edge. Else, discard it.

  Repeat step#2 until there are (V-1) edges in the spanning tree.

The algorithm is a Greedy Algorithm. The Greedy Choice is to pick the smallest weight edge that does not cause a cycle in the MST constructed so far.

Let us understand it with an example: Consider the below input graph.



The graph contains 9 vertices and 14 edges. So, the minimum spanning tree formed will be having (9

– 1) = 8 edges.

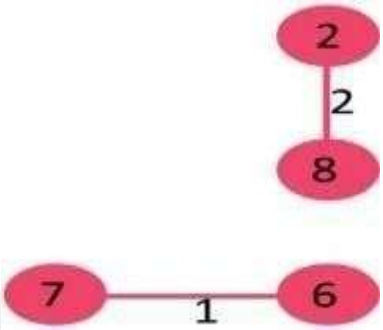
After sorting: Weight Src Dest

1	7	6
2	8	2
2	6	5
4	0	1
4	2	5
6	8	6
7	2	3
7	7	8
8	0	7
8	1	2
9	3	4
10	5	4
11	1	7
14	3	5

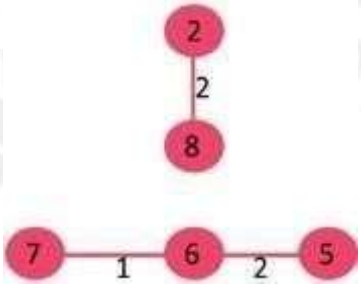
Now pick all edges one by one from sorted list of edges

- Pick edge 7-6: No cycle is formed, include it.

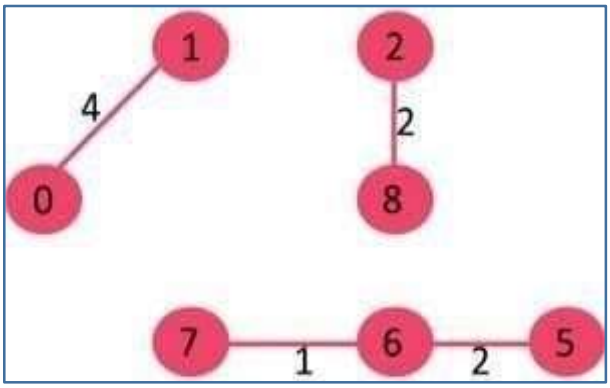
- Pick edge 8-2: No cycle is formed, include it.



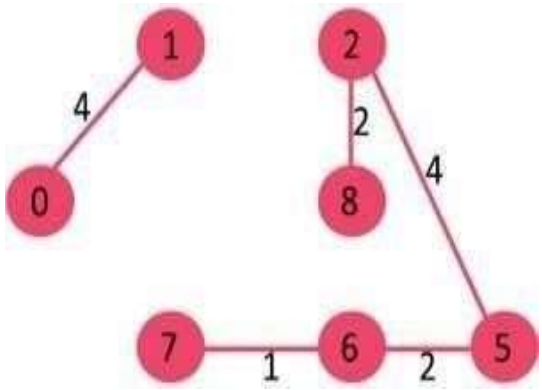
- Pick edge 6-5: No cycle is formed, include it.



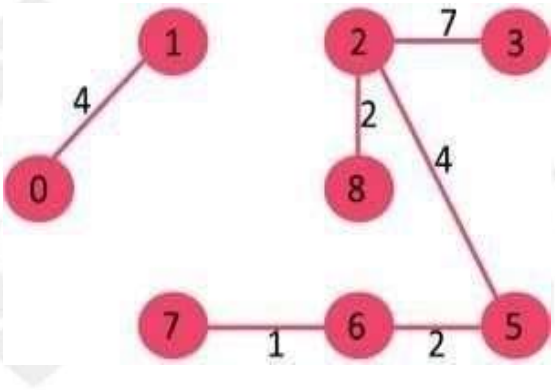
- Pick edge 0-1: No cycle is formed, include it.



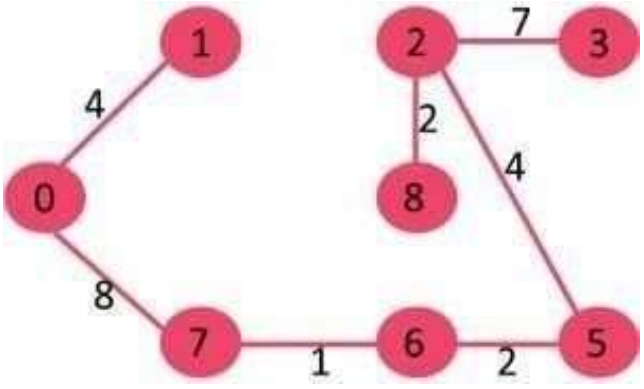
- Pick edge 2-5: No cycle is formed, include it.



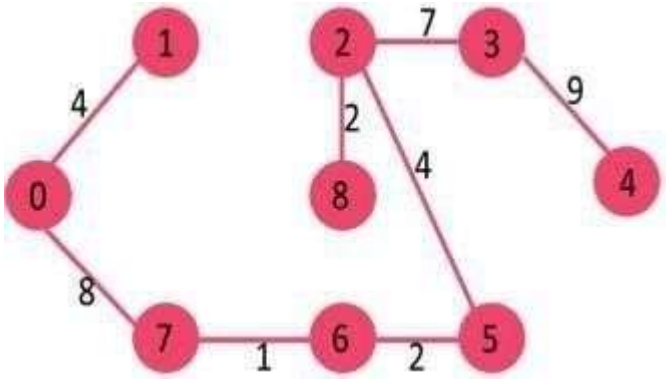
- Pick edge 8-6: Since including this edge results in cycle, discard it.
- Pick edge 2-3: No cycle is formed, include it.



- Pick edge 7-8: Since including this edge results in cycle, discard it.
- Pick edge 0-7: No cycle is formed, include it



- Pick edge 1-2: Since including this edge results in cycle, discard it.
- Pick edge 3-4: No cycle is formed, include it.



Since the number of edges included equals (V – 1), the algorithm stops here.

Prim's Algorithm:

The idea behind Prim's algorithm is simple, a spanning tree means all vertices must be connected. So the two disjoint subsets (discussed above) of vertices must be connected to make a Spanning Tree. And they must be connected with the minimum weight edge to make it a Minimum Spanning Tree

Algorithm

1) Create a set mstSet that keeps track of vertices already included in MST.

2)Assign a key value to all vertices in the input graph. Initialize all key values as INFINITE. Assign key value as 0 for the first vertex so that it is picked first.

3) While mstSet doesn't include all vertices

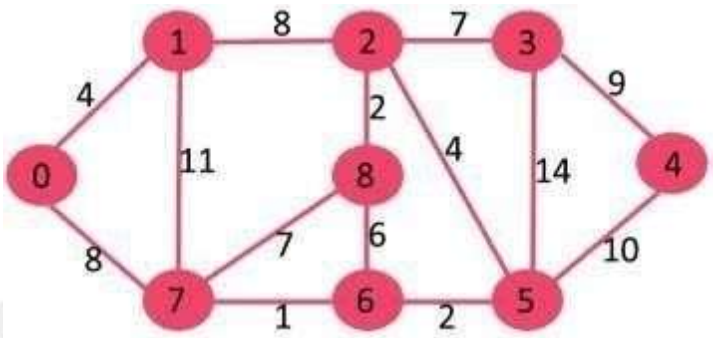
….**a)** Pick a vertex u which is not there in mstSet and has minimum key value.

….**b)** Include u to mstSet.

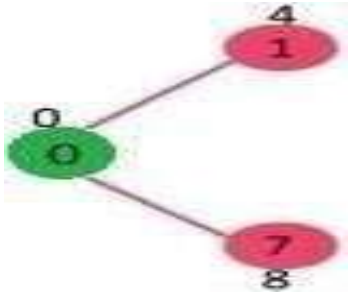
….**c)** Update key value of all adjacent vertices of u.

- To update the key values, iterate through all adjacent vertices. For every adjacent vertex v, if weight of edge u-v is less than the previous key value of v, update the key value as weight of u-v The idea of using key values is to pick the minimum weight edge from cut. The key values are used only for vertices which are not yet included in MST, the key value for these vertices indicate the minimum weight edges connecting them to the set of vertices included in MST.

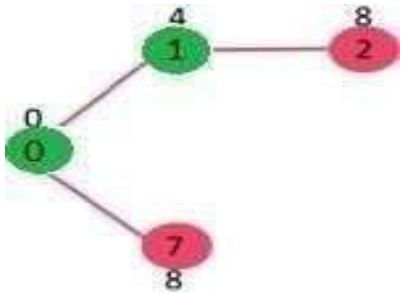
Let us understand with the following example:



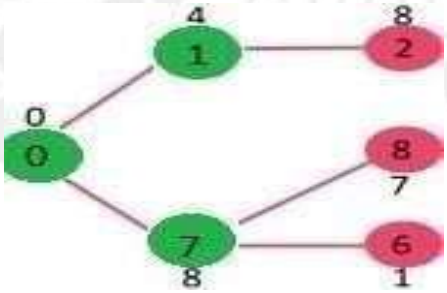
- The set mstSet is initially empty and keys assigned to vertices are {0, INF, INF, INF, INF, INF, INF, INF} where INF indicates infinite. Now pick the vertex with the minimum key value. The vertex 0 is picked , include it in mstSet. So mstSet becomes {0}. After including to mstSet, update key values of adjacent vertices. Adjacent vertices of 0 are 1 and 7. The key values of 1 and 7 are updated as 4 and 8. Following subgraph shows vertices and their key values, only the vertices with finite key values are shown.
- The vertices included in MST are shown in green color.



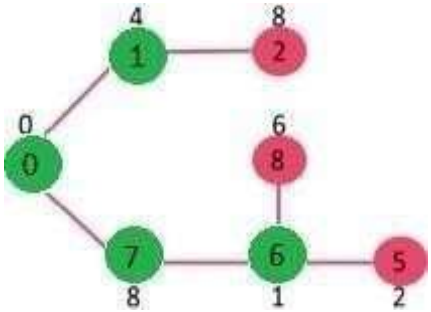
- Pick the vertex with minimum key value and not already included in MST (not in mstSET). The vertex 1 is picked and added to mstSet. So mstSet now becomes {0, 1}. Update the key values of adjacent vertices of 1. The key value of vertex 2 becomes 8.



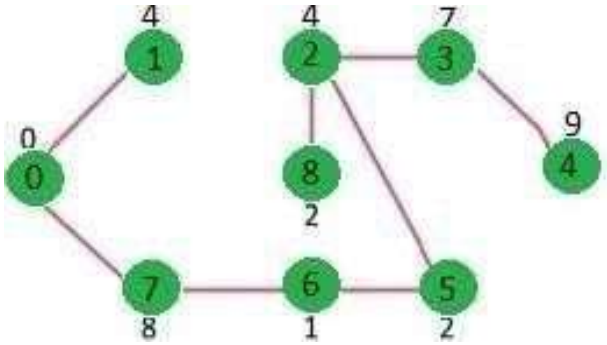
- Pick the vertex with minimum key value and not already included in MST (not in mstSET). We can either pick vertex 7 or vertex 2, let vertex 7 is picked. So mstSet now becomes {0, 1, 7}. Update the key values of adjacent vertices of 7. The key value of vertex 6 and 8 becomes finite (1 and 7 respectively).



- Pick the vertex with minimum key value and not already included in MST (not in mstSET). Vertex 6 is picked. So mstSet now becomes {0, 1, 7, 6}. Update the key values of adjacent vertices of 6. The key value of vertex 5 and 8 are updated.



- We repeat the above steps until mstSet includes all vertices of given graph. Finally, we get the following graph.



- **The following table shows the step by step procedure of finding the minimum spanning tree.**

STEP 1

Vertex	Visit	Cost	Path
0 **	**T**	**0**	**-**
1	F	INF	-
2	F	INF	-
3	F	INF	-
4	F	INF	-
5	F	INF	-
6	F	INF	-
7	F	INF	-
8	F	INF	-

STEP 2

Vertex	Visit	Cost	Path
0	**T**	**0**	**-**
1 **	**T**	**4**	**0**
2	F	INF	-
3	F	INF	-
4	F	INF	-
5	F	INF	-
6	F	INF	-
7	F	8	-
8	F	INF	-

STEP 3

Vertex	Visit	Cost	Path
0	**T**	**0**	**-**
1	**T**	**4**	**0**
2	F	8	1
3	F	INF	-
4	F	INF	-
5	F	INF	-
6	F	INF	-
7 **	**T**	**8**	**0**
8	F	INF	-

STEP 4

Vertex	Visit	Cost	Path
0	**T**	**0**	**-**
1	**T**	**4**	**0**
2	F	8	1
3	F	INF	-
4	F	INF	-
5	F	INF	-
6 **	**T**	**1**	**7**
7	**T**	**8**	**0**
8	F	7	7

STEP 5

Vertex	Visit	Cost	Path
0	**T**	**0**	**-**
1	**T**	**4**	**0**
2	F	8	1
3	F	INF	-
4	F	INF	-
5 **	**T**	**2**	**6**
6	**T**	**1**	**7**
7	**T**	**8**	**0**
8	F	6	6

STEP 6

Vertex	Visit	Cost	Path
0	**T**	**0**	**-**
1	**T**	**4**	**0**
2 **	**T**	**4**	**5**
3	F	14	5
4	F	10	5
5	**T**	**2**	**6**
6	**T**	**1**	**7**
7	**T**	**8**	**0**
8	F	6	6

STEP 7

Vertex	Visit	Cost	Path
0	T	0	-
1	T	4	0
2	T	4	5
3	F	7	2
4	F	10	5
5	T	2	6
6	T	1	7
7	T	8	0
8 **	T	2	2

STEP 8

Vertex	Visit	Cost	Path
0	T	0	-
1	T	4	0
2	T	4	5
3 **	T	7	2
4	F	10	5
5	T	2	6
6	T	1	7
7	T	8	0
8	T	2	2

STEP 9

Vertex	Visit	Cost	Path
0	T	0	-
1	T	4	0
2	T	4	5
3	T	7	2
4 **	T	9	4
5	T	2	6
6	T	1	7
7	T	8	0
8	T	2	2

9. LECTURE NOTES : UNIT – IV

Animation Videos

- Graph Traversal
- Minimum Spanning Tree Shortest path Algorithms
- Topological Sort Topological DFS
- Floyd-Warshall All Pair Shortest Algorithm

Quiz Link

Graph Quiz

10. ASSIGNMENTS : UNIT – IV

1. Given a set of vertices V in a weighted graph where its edge weights w(u, v) can be negative, find the shortest path weights d(s, v) from every source s for all vertices v present in the graph. If the graph contains a negative-weight cycle, report it. (CO4, K1)

2. Given an undirected graph, check if it is k–colorable or not and print all possible configurations of assignment of colors to its vertices. (CO4, K2)

3. Given a chessboard, find the shortest distance (minimum number of steps) taken by a knight to reach a given destination from a given source. (CO4, K3)
For example.

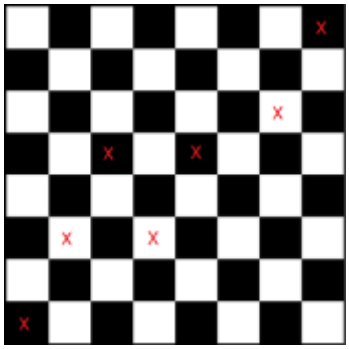
Input:

N = 8 (8 × 8 board)

Source = (7, 0)

Destination = (0, 7)

Output: Minimum number of steps required is 6



4. Given a graph which represents a flow network where every edge has a capacity. Also, given two vertices source 's' and sink 't' in the graph, find the maximum possible flow from s to t with the following constraints:

•Flow on an edge doesn't exceed the given capacity of the edge.

•Incoming flow is equal to outgoing flow for every vertex except s and t.

(CO4, K4)

5. The city has setup a radio station to broadcast the current traffic situation to all the cars moving in the city. So the cars now intelligently choose the shortest path to their destination by considering the current traffic situation. At each station, the car decides it's next hop based on it's current shortest path to the destination. The shortest path is computed by taking into account the current traffic situation. Choose the right data structure to model the problem and find the shortest path to help people to reach their destination quickly.

Note: Assume your own data for number of places in the city and distance between each places in the city. (CO4, K5)

Part A

Question & Answer

11. PART A : Q & A : UNIT – IV

SNo	Questions and Answers	CO	K
1	**Define Graph and its closely related terms.** A ***graph*** G = (V, E) consists of a set of vertices, V, and a set of edges, E. Each edge is a pair (v,w), where v,w € V. Edges are sometimes referred to as arcs. If the pair is ordered, then the graph is ***directed***. Directed graphs are sometimes referred to as digraphs. Vertex w is adjacent to v if and only if (v,w) € E. In an undirected graph with edge (v,w), and hence (w,v), w is adjacent to v and v is adjacent to w. Sometimes an edge has a third component, known as either a weight or a cost.	CO4	K1
2	**What is path and length in graph?** A path in a graph is a sequence of verices w1, w2, w3, . . . , wn such that (wi, wi+i) E for 1 i < n. The length of such a path is the number of edges on the path, which is equal to n - 1.		K1
3	**What is loop and simple path in graph?** If the graph contains an edge (v,v) from a vertex to itself, then the path v, v is sometimes referred to as a loop. The graphs we will consider will generally be loopless. A simple path is a path such that all vertices are distinct, except that the first and last could be the same.		K1
4	**What do you mean by cycle in graph?** A cycle in a directed graph is a path of length at least 1 such that w1 = wn; this cycle is simple if the path is simple.		K1
5	**Define connected, strongly connected, weakly connected graph.** An undirected graph is ***connected*** if there is a path from every vertex to every other vertex. A directed graph with this property is called ***strongly connected***. If a directed graph is not strongly connected, but the underlying graph (without direction to the arcs) is connected, then the graph is said to be ***weakly connected***.		K1

11. PART A : Q & A : UNIT – IV

SNo	Questions and Answers	CO	K
12	**What do you mean by indegree and outdegree?** A directed graph with vertices labeled (indegree, outdegree). For a vertex, the number of head ends adjacent to a vertex is called the indegree of the vertex and the number of tail ends adjacent to a vertex is its outdegree (called "branching factor" in trees).	CO4	K1
13	**Give any two applications of DFS and BFS.** DFS : Detecting cycle in graph, Topological sorting, Path finding BFS : GPS Navigation System, Broadcasting in Network, Minimum spanning tree.		K2
14	**Define depth-first traversal in a tree.** Depth first works by selecting one vertex V of G as a start vertex ; V is marked visited. Then each unvisited vertex adjacent to V is searched in turn using depth first search recursively. This process continues until a dead end (i.e) a vertex with no adjacent unvisited vertices is encountered		K1
15	**List the two key points of DFS** If path exists from one node to another, walk across the edge – exploring the edge. If path does not exist from one specific node to any other node, return to the previous node where we have been before – backtracking		K2
16	**Define biconnectivity and articulation points.** A connected undirected graph is biconnected if there are no vertices whose removal disconnects the rest of the graph. If a graph is not biconnected, the vertices whose removal would disconnect the graph are known as articulation points.		K1
17	**What is minimum spanning tree?** A minimum spanning tree of a weighted connected graph G is its spanning tree of the smallest weight, where the weight of a tree is defined as the sum of the weights on all its edges.		K1

11. PART A : Q & A : UNIT – IV

SNo	Questions and Answers	CO	K
18	**Name the well-known algorithms used to construct a minimum spanning tree.** Kruskal's algorithm : A greedy algorithm in graph theory that finds a minimum spanning tree for a connected weighted graph. It finds a subset of the edges that forms a tree that includes every vertex, where the total weight of all the edges in the tree is minimized. Prim's algorithm : A greedy algorithm that finds a minimum spanning tree for a weighted undirected graph. This means it finds a subset of the edges that forms a tree that includes every vertex, where the total weight of all the edges in the tree is minimized.	CO4	K1
19	**What is all pair shortest path in graph?** Given a directed, connected weighted graph G(V,E), for each edge ⟨u,v⟩∈E, a weight w(u,v) is associated with the edge. The all pairs of shortest paths problem (APSP) is to find a shortest path from u to v for every pair of vertices u and v in V.		K1
20	**What is single source shortest path in graph?** Given a directed graph G = (V,E), with non-negative costs on each edge, and a selected source node v in V, for all w in V, find the cost of the least cost path from v to w. The cost of a path is simply the sum of the costs on the edges traversed by the path.		K1
21	**Define Shortest path.** In graph theory, the shortest path problem is the problem of finding a path between two vertices (or nodes) in a graph such that the sum of the weights of its constituent edges is minimized.		K1
22	**What do you mean by tree edge and back edge?** If w is undiscovered at the time vw is explored, then vw is called a tree edge and v becomes the parent of w. If w is the ancestor of v, then vw is called a back edge.		K1

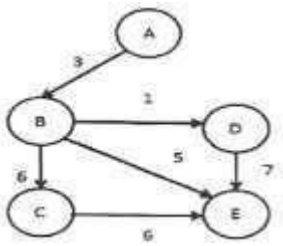
11. PART A : Q & A : UNIT – IV

SNo	Questions and Answers	CO	K
23	**What is an undirected acyclic graph?** When every edge in an acyclic graph is undirected, it is called an undirected acyclic graph. It is also called as undirected forest.	CO4	K1
24	**Compare Prim and Kruskal algorithm.** **Kruskal** Begins with forest and merge into a tree Has better running times if the number of edges is low **Prim** Always stays as a tree Has a better running time if both the number of edges and the number of nodes are low.		K1
25	**Write down the methods used to find the articulation points.** Brute-Force approach DFS based approach		K1

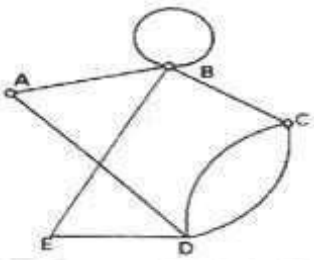
12. PART B QUESTIONS : UNIT – IV

(CO3, K2 / K3)

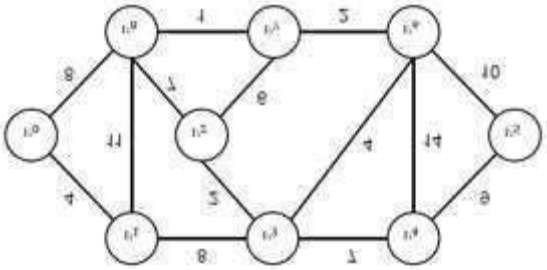
1. Apply an appropriate algorithm to find the shortest path from the node 'A' to every other nodes in the given graph. (13)



2. Explain in detail about strongly connected components and illustrate with example. (7)
3. Find an Euler path or an Euler circuit using DFS for the given graph. (6)



4. Distinguish between breadth first search and depth first search with suitable example. (13)
5. State and explain topological sorting with an example. (10)
6. With necessary diagram discuss the different ways of representing a graph. (7)
7. Explain how to find the articulation points? Give example. List out the articulation points in the example using appropriate algorithm.
8. List out the methods to calculate minimum spanning tree. Explain any one method with example.
9. Apply prim's algorithm for the following graph and find out the minimum spanning tree and its cost.



13. SUPPORTIVE ONLINE CERTIFICATION COURSES

UNITS : I TO V

COURSERA

DATA STRUCTURES

DATA STRUCTURES AND ALGORITHMS

DATA STRUCTURES AND PERFORMANCE

PYTHON DATA STRUCTURES

UDEMY

LEARNING DATA STRUCTURES IN C PROGRAMMING LANGUAGE

DATA STRUCTURES IN C LANGUAGE

NPTEL

PROGRAMMING, DATA STRUCTURES AND ALGORITHM USING PYTHON

14. REAL TIME APPLICATIONS : UNIT - IV

NON LINEAR DATA STRUCTURES - GRAPHS

Few important real life applications of graph data structures are:

- Representing networks and routes in communication, transportation and travel applications
- Routes in GPS
- Interconnections in social networks and other network based applications
- Mapping applications
- Ecommerce applications to present user preferences
- Utility networks to identify the problems posed to municipal or local corporations
- Resource utilization and availability in an organization
- Document link map of a website to display connectivity between pages through hyperlinks
- Robotic motion and neural networks

1. Facebook: Each user is represented as a vertex and two people are friends when there is an edge between two vertices. Similarly friend suggestion also uses graph theory concept.
2. Google Maps: Various locations are represented as vertices and the roads are represented as edges and graph theory is used to find shortest path between two nodes.
3. Recommendations on e-commerce websites: The "Recommendations for you" section on various e-commerce websites uses graph theory to recommend items of similar type to user's choice.
4. Graph theory is also used to study molecules in chemistry and physics.
5. Lots of people look at stock market graphs on a daily basis, for one example. A weather 'map' can graph of temperature data (or precipitation data), where colors represent certain levels of the data
6. Graph is powerful and versatile data structure that easily allow to you represent real life relationships between different type of data nodes.

a) They include , study of molecule construction in bond of chemistry and the study of atoms.

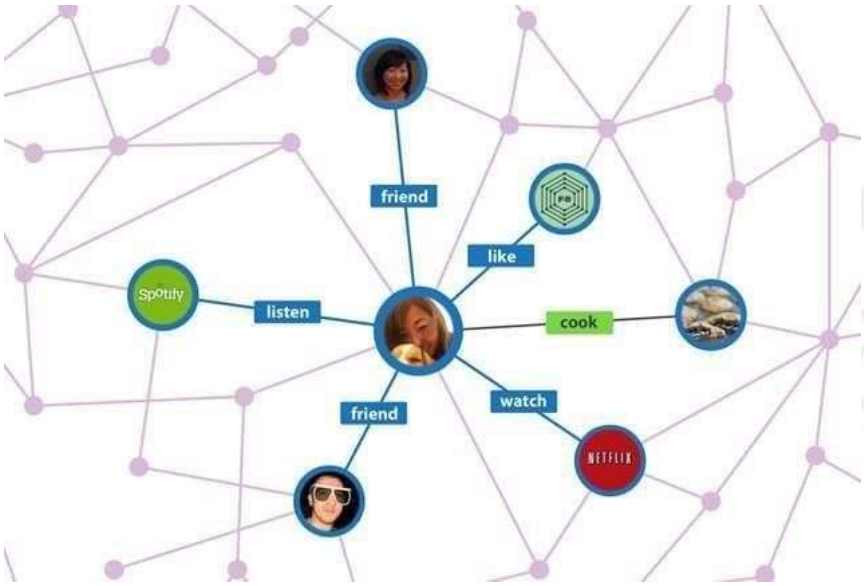
b) Graph theoretical concept are widely used in operation research .

c) Vertical nodes where the data stored i.e. the no of images on the left.

d) The edges connection which connect nodes lines between the no of the images.

Social Graphs

Social graphs draw edges between you and the people, places and things you interact with online



Facebook's Graph API

Facebook's Graph API is perhaps the best example of application of graphs to real life problems. The Graph API is a revolution in large-scale data provision

- **On The Graph API, everything is a vertice or node.** This are entities such as Users, Pages, Places, Groups, Comments, Photos, Photo Albums, Stories, Videos, Notes, Events and so forth. Anything that has properties that store data is a vertice.

- **And every connection or relationship is an edge.** This will be something like a User posting a Photo, Video or Comment etc., a User updating their profile with a their Place of birth, a relationship status Users, a User liking a Friend's Photo etc.

- The Graph API uses this collections of vertices and edges (essentially graph data structures) to store its data. The Graph API is also a GraphQL API. This is the language it uses to build and query the schema.
- The Graph API has come into some problems because of it's ability to obtain unusually rich info about user's friends.
- **Knowledge Graphs**
- Google's Knowledge Graph
- A knowledge graph has something to do with linking data and graphs...some kind of graph-based representation of knowledge. It still isn't what is can and can't do yet.
- https://youtu.be/mmQl6VGvX-c
- **Recommendation Engines**
- Yelp's Local Graph
- Yelps has been slowly phasing out their old Fusion API for a GraphQL API.
- The Local Graph API promises to make it easier for developers to integrate Yelp's data and share great local businesses through their apps

14. REAL TIME APPLICATIONS : UNIT - IV

- **GraphQL leverages** the power of **graph data structures** by modeling the business problem as a graph **within its schema**.
- The most common use case for GraphQL is operating on graph data structures. Both Apollo Client and Relay operate on GraphQL data as a normalized graph.
- **On the Local Graph API, Yelp represents your business as a vertice** with name, id, alias, is_claimed, is_closed etc. graph properties.
- **Yelp also creates additional vertices for Place** (as custom type Location in GraphQL schema,), **Categories** (as custom type Category in GraphQL schema), **Review** (as type Review) and **Hours** (as type Hours)
- **Yelp creates edges** with relationships such as the location of a business with a certain name, the opening hours of a business, the reviews of a business, the category of a business.
- Using the local graph feature, a **yelp app can uses your location to match recommendations of businesses close to you**. In this case your location and the location of the business are both vertices while the recommendation is the edge.
- **Path Optimization Algorithms**
- Path optimizations are primarily occupied with **finding the best connection that fits some predefined criteria** e.g. speed, safety, fuel etc **or set of criteria** e.g prodecures, routes.
- In unweighted graphs, the **Shortest Path of a graph is the path with the least number of edges**. Breadth First Search (BFS) is used to find the shortest paths in graphs—we always reach a node from another node in the fewest number of edges in breadth graph traversals
- Any Spanning Tree is a Minimum Spanning Tree unweighted graphs using either BFS or Depth First Search.
- Google Maps Platform (Maps, Routes APIs)

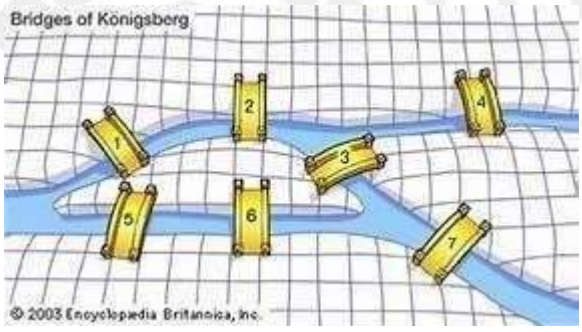
14. REAL TIME APPLICATIONS : UNIT - IV

- **Google Maps and Routes APIs are classic Shortest Path APIs**. This a graph problem that's very easy to solve with edge-weighted directed graphs (digraphs).
- The idea of a Map API is to **find the shortest path from one vertex to every other** as in a single source shortest path variant, from your current location to every other destination you might be interested in going to on the map.
- The idea of by contrast Routing API to **find the shortest path from one vertex to another** as in a source sink shortest path variant, from s to t.
- Shortest Path APIs are typically directed graphs. The underlying data structures and graphy too.
- **Flight Networks**
- For flight networks, **efficient route optimizations perfectly fit graph data strutures**. Using graph models, airport procedures can be modeled and optimized efficiently.
- Computing best connections in flight networks is a key application of algorithm engineering.
- In flight network, graph data strutures are used to compute shortest paths and fuel usage in route planning, often in a multi-modal context.
- The **vertices in flight networks are places** of departure and destination, airports, aircrafts, cargo weights.
- The **flight trajectories between airports are the edges**. Turns out it's very feasible to fit graph data strutures in route optimizations because of precompiled full distance tables between all airports.
- Entities such as flights can have properties such as fuel usage, crew pairing which can themselves be more graphs.
- GPS Navigations Systems

14. REAL TIME APPLICATIONS : UNIT - IV

- **Car navigations also use Shortest Path APIs**.
- Although this is still a type of a routing API it would differ from the Google Maps Routing API because it **is single-source** (from one vertex to every other i.e. it computes locations from where you are to any other location you might be interested in going.)
- BFS is used to find all neightbouring locations.
- **EULER CIRCUIT APPLICATION**
- **KOINSBERG BRIDGE PROBLEM**
- The origin of graph theory was in the times of Euler. He first used graph theory as a method to solve the koinsberg bridge problem.

The problem is given seven bridges, is it possible to cross through all the bridges such that you cross through a bridge only once.



He solved the problem by modelling each ladmass as a vertex and a bridge between them as an edge. He noted that while crossing a bridge you leave one land mass and come on to another and therefore if you have to enter and exit a landmass such that you don't repeat the bridge then the number of bridges connecting that landmass should be even.

- In the above problem every vertex had odd number of edges therefore it was impossible to have a walk such that every bridge is touched upon only once.

14. REAL TIME APPLICATIONS : UNIT - IV

- **TOPOLOGICAL SORT APPLICATION**
- **Build systems**
- Consider a source code structure where you are building several libraries (DLLs) and they have dependencies on each other. For example, to build dll A, you must have built DLLs B, C an D (Maybe you have a reference of B,C and D in the project that builds A).
- Let's mark a dependency edge from each of B, C and D to A implying that A depends on the other three and can only be built once each of the three are built. Technically speaking, (u,v) => An edge from u to v implies v.dll can be built only when u.dll is already built.
- After constructing a graph of these dlls and dependency edges, you can conclude that a successful build is possible iff the resulting graph is acyclic. How does the build system decide in which order to build these dlls? It sorts them topologically.
- Therefore, in an order like X->Z->T->B->D->C->A, it can start building X (which may only depend on external assemblies already built), then follow the topologically sorted list of assemblies.

15. CONTENTS BEYOND SYLLABUS : UNIT – IV

All-Pairs Shortest Paths Algorithms – Applications of Graphs

- The all pair shortest path algorithm is also known as Floyd-Warshall algorithm is used to find all pair shortest path problem from a given weighted graph. As a result of this algorithm, it will generate a matrix, which will represent the minimum distance from any node to all other nodes in the graph.
- At first the output matrix is same as given cost matrix of the graph. After that the output matrix will be updated with all vertices k as the intermediate vertex.
- The time complexity of this algorithm is O(V3), here V is the number of vertices in the graph.

Detailed description can be obtained from the link,
https://iq.opengenus.org/floyd-warshall-algorithm-shortest-path-between-all-pair-of-nodes/#algorithm

16. ASSESSMENT SCHEDULE

Tentative schedule for the Assessment During 2023-2024

Even semester

S.NO	Name of the Assessment	Start Date	End Date	Portion
1	Unit Test 1			UNIT 1
2	IAT 1	26.02.2024	02.03.2024	UNIT 1 & 2
3	Unit Test 2			UNIT 3
4	IAT 2	11.04.2024	17-04-2024	UNIT 3 & 4
5	Revision 1			UNIT 5 , 1 & 2
6	Revision 2			UNIT 3 & 4
7	Model	02-05-2024	11-05-2024	ALL 5 UNITS

Supportive Online Certification

Unit IV

Certification Courses

1. Coursera Courses:

Name of the Course: Data Structures

https://www.coursera.org/lecture/data-structures/complete-binary-trees-gl5Ni?utm_source=gg&utm_medium=sem&utm_campaign=B2C_INDIA_google-it-automation_FTCOF_professional-certificates_arte-re-PMAX_non-nrl_after14D&utm_content=B2C&campaignid=19201532531&adgroupid=&device=c&keyword=&matchtype=&network=x&devicemodel=&adpostion=&creativeid=&hide_mobile_promo&gclid=EAIaIQobChMIpbrK3NHs_gIVmbKWCh1DGwynEAAYAiAAEgJLKPD_BwE

2. Udemy Courses:

Name of the Course: Complete course on Tree - Data Structures

https://www.udemy.com/course/binary-trees-crash-course/

3. Infosys Spring Board

Name of the course: Introduction to Binary Trees, Sorting Overview

https://infyspringboard.onwingspan.com/web/en/app/toc/lex_auth_01350157816505139210584/

Real-time Applications

Unit IV

R.M.K
GROUP OF
INSTITUTIONS

Prescribed Text book & References

Unit IV

TEXT BOOKS:

- Mark Allen Weiss, "Data Structures and Algorithm Analysis in C++", 4th Edition, Pearson Education, 2014.
- Sartaj Sahni, "Data Structures, Algorithms and Applications in C++", Silicon paper publications, 2004.

REFERENCES:

- Rajesh K. Shukla, "Data Structures using C and C++", Wiley India Publications, 2009.
- Narasimha Karumanchi, "Data Structure and Algorithmic Thinking with Python: Data Structure and Algorithmic Puzzles", CareerMonk Publications, 2020.
- Jean-Paul Tremblay and Paul Sorenson, "An Introduction to Data Structures with Application", McGraw-Hill, 2017.
- Mark Allen Weiss, "Data Structures and Algorithm Analysis in Java", Third Edition, Pearson Education, 2012.
- Ellis Horowitz, Sartaj Sahni, Susan Anderson-Freed, "Fundamentals of Data Structures in C", Second Edition, University Press, 2008.
- Ellis Horowitz, Sartaj Sahni, Dinesh P Mehta, "Fundamentals of Data Structures in C++", Second Edition, Silicon Press, 2007.
- https://infyspringboard.onwingspan.com/web/en/app/toc/lex_auth_013501578165051392105 84/overview

Mini Project Suggestions

Unit IV

R.M.K
GROUP OF
INSTITUTIONS

Mini Project Suggestions

1.Cash Flow Minimizer (Graphs)

•Cash flow is the net balance of cash moving into or out of business at a specific time.
•An application cash flow minimizer can be built using Graphs concepts to minimize this cash flow.
•For example, Assume three friends: friend one, friend two, and friend three. Friend one has to give 4k to friend three and 2k to friend one.
•And friend two has to give 3k to friend three. Now minimize the flow between friend one and friend two by direct payment to friend three by friend one.
•An application can be created with the same mechanism for translation

2.Map Navigator(Dijkstra's Algorithm)

•Everyone is well-known for Google Maps. Google map is an efficient application showing the shortest path from source to destination.
•Google Maps application is built using Dijkstra's algorithm.
•We can build the same application for the metro using Dijkstra's algorithm
•Dijkstra's algorithm efficiently finds the minimum distance between nodes using the edge values.
•We can consider the stations as nodes and distance as edges; by using Dijkstra's algorithm, we can find the shortest time path and shortest cost path.

3.Maze Generator

There are many ways to create maze games, we can create a maze generator only (which just generates the maze) or we can create a fully functional maze game where we can control a *sprite* for example to navigate through the maze.

To create the maze itself we can use a **graph** and for the maze navigation (to find the shortest path from the entrance to the exit) we can use the **breadth-first search** (BFS) algorithm. While using this algorithm, we can keep track of the parent node of each visited node. This allows us to reconstruct the path from the exit back to the entrance once the destination is reached.

4.Graph Visualization

Build a tool for visualizing graphs and their properties and represent and manipulate graphs, add options to customize the appearance of the graphs.

5. Social Media Analytics

Design a tool that analyses social media data and provides insights. Utilize graphs to represent social connections, and add features for sentiment analysis and content scheduling.

Thank you

Disclaimer: